DATA SCIENCE 2VG3 Homework 0 2025/2026
Due: End of Day, Thursday 22th January 2026, via Avenue

Python Warm-up

For this homework you will write short snippet of Python to solve three exercises. You will implement your code with the HW0.py code provided on Avenue. When you run the code, it will tell you if it is working correctly for each case by saying 'OK' for each test. Once it works, upload it to avenue for credit.

Exercise 1 match_ends

Given a list of strings, count of the number of strings where the string length is 2 or more and the first and last chars of the string are the same, e.g. 'abba', 'dd', 'sos' count, 'a', 'soso', 'abc' don't count

Note: python does not have a ++ operator, but += works.

```
def match_ends(words):
  #your code here
  return count
```

Exercise 2 front_x

Given a list of strings, return a list with the strings in sorted order, except group all the strings that begin with 'x' first.
e.g. ['mix', 'xyz', 'apple', 'xanadu', 'aardvark'] yields ['xanadu', 'xyz', 'aardvark', 'apple', 'mix']

Hint: this can be done by making 2 lists and sorting each of them before combining them.

```
def front_x(words):
  #your code here
  return combinedlist
```

Exercise 3 sort_last

Given a list of non-empty tuples, return a list sorted in increasing order by the last element in each tuple.
e.g. [(1, 7), (1, 3), (3, 4, 5), (2, 2)] yields [(2, 2), (1, 3), (3, 4, 5), (1, 7)]

Hint: use a custom key= function to extract the last element form each tuple.

```
def sort_last(tuples):
  # your code here
  return sortedlist
```

```python
#!/usr/bin/python3

# 1. match_ends
# Given a list of strings, count of the number of strings where the string
# length is 2 or more and the first and last chars of the string are the same,
# e.g.  ‘abba’,  ‘dd’,   ‘sos’  count,    ‘a’, ‘soso’, ‘abc’  don’t count
# Note: python does not have a ++ operator, but += works.
def match_ends(words):
    # your code here
    return


# 2. front_x
# Given a list of strings, return a list with the strings
# in sorted order, except group all the strings that begin with 'x' first.
# e.g. ['mix', 'xyz', 'apple', 'xanadu', 'aardvark'] yields
# ['xanadu', 'xyz', 'aardvark', 'apple', 'mix']
# Hint: this can be done by making 2 lists and sorting each of them
# before combining them.
def front_x(words):
    # your code here
    return


# 3. sort_last
# Given a list of non-empty tuples, return a list sorted in increasing
# order by the last element in each tuple.
# e.g. [(1, 7), (1, 3), (3, 4, 5), (2, 2)] yields
# [(2, 2), (1, 3), (3, 4, 5), (1, 7)]
# Hint: use a custom key= function to extract the last element form each tuple.
def sort_last(tuples):
    # your code here
    return


# Simple provided test() function used in main() to print
# what each function returns vs. what it's supposed to return.
def test(got, expected):
  if got == expected:
    prefix = ' OK '
  else:
    prefix = '  X '
  print('%s got: %s expected: %s' % (prefix, repr(got), repr(expected)))


# Calls the above functions with interesting inputs.
def main():
  print('match_ends')
  test(match_ends(['aba', 'xyz', 'aa', 'x', 'bbb']), 3)
  test(match_ends(['', 'x', 'xy', 'xyx', 'xx']), 2)
  test(match_ends(['aaa', 'be', 'abc', 'hello']), 1)
```

```
  print()
  print('front_x')
  test(front_x(['bbb', 'ccc', 'axx', 'xzz', 'xaa']),
       ['xaa', 'xzz', 'axx', 'bbb', 'ccc'])
  test(front_x(['ccc', 'bbb', 'aaa', 'xcc', 'xaa']),
       ['xaa', 'xcc', 'aaa', 'bbb', 'ccc'])
  test(front_x(['mix', 'xyz', 'apple', 'xanadu', 'aardvark']),
       ['xanadu', 'xyz', 'aardvark', 'apple', 'mix'])


  print()
  print('sort_last')
  test(sort_last([(1, 3), (3, 2), (2, 1)]),
       [(2, 1), (3, 2), (1, 3)])
  test(sort_last([(2, 3), (1, 2), (3, 1)]),
       [(3, 1), (1, 2), (2, 3)])
  test(sort_last([(1, 7), (1, 3), (3, 4, 5), (2, 2)]),
       [(2, 2), (1, 3), (3, 4, 5), (1, 7)])


if __name__ == '__main__':
  main()
```